

# BAŞLIK: POLİTİKA GRADYANI (POLICY GRADIENT) VE DERİN PEKİŞTİRMELİ ÖĞRENME (DQN)

TÜR: Akademik/Teknik Detaylı Anlatım

## BÖLÜM 1: NEDEN POLİTİKA TABANLI YÖNTEMLERE İHTİYAÇ VAR?

21.2.1'de gördüğümüz Q-Learning, her (durum, eylem) çifti için ayrı bir değer tuttuğu için, durum uzayı büyüdükçe pratik olmaktan çıkar. CartPole ortamını ele alalım: durum, 4 gerçek sayıdan oluşur (arabanın konumu, hızı, direğin açısı, direğin açısal hızı). Bu 4 boyutlu uzayda sonsuz sayıda olası durum vardır — bir Q-tablosunda her birine bir satır ayıramayız.

Politika tabanlı (policy-based) yöntemler bu soruna farklı bir açıdan yaklaşır: bir ara değer tablosu öğrenmek yerine, politikayı  $\pi(a|s)$  doğrudan parametrik bir fonksiyon (örn. ağırlıkları  $\theta$  olan küçük bir yapay sinir ağı ya da hatta tek katmanlı bir doğrusal model) olarak temsil eder ve bu ağırlıkları doğrudan optimize eder. Fonksiyon parametrik olduğu için, daha önce hiç görülmemiş bir durumla karşılaşıldığında bile (durumlar sürekli olduğu için pratikte her durum "yeni"dir) makul bir eylem dağılımı üretebilir — tıpkı bir doğrusal regresyon modelinin, eğitim setinde hiç görmediği bir x değeri için de bir y tahmini üretebilmesi gibi.

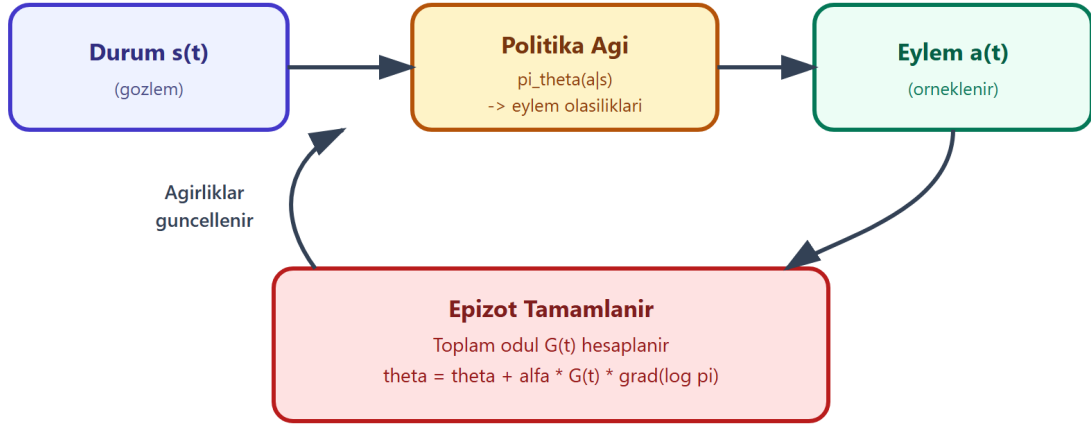
### 1.1. Stokastik Politika: Olasılık Dağılımı Olarak Eylem Seçimi

Q-Learning'de eylem seçimi "en yüksek Q-değerine sahip eylemi seç" şeklinde katı (deterministic) bir kuraldı. Politika gradyanı yöntemlerinde ise  $\pi(a|s)$  doğrudan bir olasılık dağılımı üretir (örneğin CartPole'da: "%70 ihtimalle sağa, %30 ihtimalle sola"). Eylem bu dağılımdan örneklenerek (sample) seçilir. Bunun doğal bir sonucu vardır: keşif (exploration),  $\epsilon$ -greedy gibi ek bir mekanizma gerektirmeden, politikanın kendisine gömülü olarak gelir — olasılık dağılımı ne kadar "yayvansa" (dağınıksa) o kadar fazla keşif yapılır, dağılım keskinleştikçe (bir eylemde yoğunlaştıkça) sömürüye kayar.

## BÖLÜM 2: REINFORCE ALGORİTMASI (MONTE CARLO POLİTİKA GRADYANI)

REINFORCE, en basit ve en temel politika gradyanı algoritmasıdır (Williams, 1992) ve fikrini üç cümlede özetleyebiliriz: bir epizot baştan sona oynanır; epizot sonunda her adımda alınan eylemin, epizotun toplam getirisine ( $G_t$ ) ne kadar katkı sağladığı hesaplanır; getirisi yüksek epizotlarda seçilen eylemlerin olasılığı artırılır, düşük getirili epizotlarda seçilenlerinki azaltılır.

## Policy Gradient (REINFORCE) Öğrenme Dongusu



İyi sonuçlanan (yüksek ödüllü) eylemlerin olasılığı artırılır, kötü sonuçlananlar azaltılır.

Şekil 3 — Politika ağı, durumdan eylem olasılıkları üretir; epizot bittiğinde toplam ödüle göre ağırlıklar güncellenir.

### 2.1. Log-Türev Hilesi ve Gradyan Formülü

Politikanın parametrelerini ( $\theta$ ), beklenen getiriye maksimize edecek yönde güncellemek istiyoruz. REINFORCE'un matematiksel türetimi (log-derivative trick) sonunda şaşırtıcı derecede sade bir güncelleme kuralına indirgenir:

$$\theta \leftarrow \theta + \alpha \cdot G_t \cdot \text{grad}_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- $\text{grad}_{\theta} \log \pi_{\theta}(a_t | s_t)$ : Seçilen eylemin olasılığını artırmak için ağırlıkların hangi yönde değişmesi gerektiğini gösteren gradyan (klasik bir sınıflandırma modelindeki log-olabilirlik gradyanıyla neredeyse aynı matematiksel yapıdadır).
- $G_t$ : O eylemden sonra epizot boyunca elde edilen toplam indirgenmiş ödül (bkz. 21.1.1, Bölüm 3.2 — aynı formül:  $G(t) = r(t+1) + \gamma \cdot r(t+2) + \dots$ ).
- Çarpımın anlamı: gradyan,  $G_t$  ile "ölçeklenir".  $G_t$  büyük (iyi bir epizot) ise o adımdaki eylemin olasılığını güçlü şekilde artır;  $G_t$  küçük ya da negatifse (kötü bir epizot) o adımdaki eylemin olasılığını azalt.

Kritik bir fark: Q-Learning her adımda (online, tek adımlık TD hatasıyla) güncelleme yaparken, REINFORCE bir epizot tamamen bitmeden güncelleme yapamaz — çünkü  $G_t$ 'yi hesaplamak için epizotun sonundaki tüm ödüllere ihtiyaç vardır. Bu yüzden REINFORCE'a "Monte Carlo" yöntemi denir: tahmini, gerçek tam bir örneklemden (epizottan) hesaplar, TD gibi bootstrapping yapmaz.

## BÖLÜM 3: DERİN PEKİŞTİRMELİ ÖĞRENME VE DQN (KAVRAMSAL)

REINFORCE, politikayı doğrudan öğrenir. Alternatif bir yol daha vardır: Q-değerini bir tablo yerine bir yapay sinir ağı ile temsil etmek. Bu yaklaşıma Derin Q-Ağı (Deep Q-Network - DQN) denir ve 2015 yılında DeepMind'ın Nature dergisinde yayımladığı makale ile, Atari 2600 oyunlarını doğrudan piksel girdisinden insan üstü seviyede oynayarak derin pekiştirmeli öğrenme alanını başlatan çalışmadır.

DQN'in temel fikri basittir:  $Q(s,a)$  tablosu yerine, girdisi durum  $s$  olan ve çıktısı her eylem için bir Q-değeri tahmini olan bir sinir ağı  $Q(s,a; \theta)$  kullanılır. Bellman güncelleme kuralı (bkz. 21.2.1, Bölüm 2) artık bir tablo hücresini güncellemek yerine, ağı  $\theta$  ağırlıklarını, hedef değerle tahmin arasındaki farkı (TD hatası) bir kayıp fonksiyonu olarak kullanıp gradyan inişiyile güncellemeye dönüşür — yani standart bir gözetimli öğrenme regresyon problemi haline gelir.

### 3.1. DQN'i Kararlı Hale Getiren İki Kritik Numara

Ham haliyle "Q-tablosunu bir sinir ağıyla değiştirmek" kararsız ve ıraksayan (diverge eden) bir eğitime yol açar, çünkü ardışık deneyimler birbiriyle güçlü şekilde korelelidir (bir CartPole epizotundaki art arda gelen durumlar neredeyse aynıdır) ve hedef değer de aynı ağıdan hesaplandığı için sürekli hareket eden bir hedefi kovalamak gibi olur. DQN makalesi bu iki sorunu şu tekniklerle çözer:

- Deneyim Tekrarı (Experience Replay): Ajanın yaşadığı  $(s, a, r, s')$  geçişleri bir hafıza havuzunda (replay buffer) biriktirilir; ağ, ardışık deneyimlerle değil, bu havuzdan rastgele örneklenen gruplarla (mini-batch) eğitilir. Bu, korelasyonu kırar ve eski deneyimlerin tekrar kullanılmasını sağlayarak veri verimliliğini artırır.
- Hedef Ağ (Target Network): Hedef değer,  $[r + \gamma \cdot \max_a Q(s',a)]$  hesaplanırken, sürekli güncellenen asıl ağ yerine, ağırlıkları belirli aralıklarla (örn. her 1000 adımda bir) dondurulup kopyalanan ayrı bir "hedef ağ" kullanılır. Bu, hedefin çok sık değişmesini engelleyerek eğitimi kararlı hale getirir.

Bu repodaki `21.3.2_dqn_cartpole.py` dosyası, pedagojik netlik ve harici derin öğrenme bağımlılığı gerektirmemesi için DQN yerine, tamamen NumPy ile yazılmış bir REINFORCE (politika gradyanı) uygulaması kullanır — ama yukarıdaki kavramsal çerçeve (fonksiyon yaklaşılama, sürekli durum uzayında genelleme) birebir aynıdır ve PyTorch/TensorFlow gibi bir çerçeveye geçtiğinizde DQN'i kurmak için gereken tüm teorik temeli oluşturur.

## BÖLÜM 4: CARTPOLE ÖRNEĞİ ÜZERİNDEN UYGULAMA

CartPole-v1 ortamında bir araba, üzerine mafsallı şekilde tutturulmuş bir direği dengede tutmaya çalışır. Durum uzayı 4 sürekli değerden oluşur (araba konumu, araba hızı, direk açısı, direk açısal hızı); eylem uzayı ise ayrıktır: arabayı sola ya da sağa it (2 eylem). Her adımda direk düşmediği sürece +1 ödül verilir; direk belirli bir açıyı aşarsa ya da araba ekrandan çıkarsa epizot biter.

`21.3.2_dqn_cartpole.py` dosyasında politika, 4 girişi (durum) alıp 2 çıkış (sol/sağ olasılığı) üreten, softmax aktivasyonlu, tek katmanlı doğrusal bir model olarak NumPy ile sıfırdan uygulanır. Eğitim döngüsü tam olarak Bölüm 2'deki REINFORCE algoritmasını izler: epizot oynanır, her adımın (durum, eylem, ödül) üçlüsü kaydedilir, epizot bitince  $G_t$  değerleri geriye doğru hesaplanır ve ağırlıklar güncellenir. Ortalama epizot uzunluğunun (direğin ayakta kalma süresinin) eğitim boyunca artması, politikanın gerçekten öğrendiğinin somut kanıtıdır.

## SONUÇ: UYGULAMA ALANLARI

Politika tabanlı ve derin pekiştirmeli öğrenme yöntemleri, tablo tabanlı Q-Learning'in sınırlarını aşarak bugün şu alanlarda hayat buluyor:

1. Sürekli Kontrol Problemleri: Robot kollarının eklem açılarını, otonom araçların direksiyon/gaz oranını sürekli (continuous) bir aralıkta belirlemesi — Q-Learning'in tablo yapısı bunu doğrudan yapamaz, politika gradyanı yöntemleri (örn. PPO, DDPG) doğrudan yapabilir.
2. Gelişmiş Oyun Yapay Zekaları: AlphaGo/AlphaZero, OpenAI Five (Dota 2), StarCraft II'de AlphaStar — hepsi politika ağı + değer ağı kombinasyonlarına dayanır.
3. Büyük Dil Modeli İnce Ayarı (RLHF): ChatGPT ve benzeri modellerin insan tercihlerine göre ince ayarlanması, PPO (Proximal Policy Optimization) adlı bir politika gradyanı algoritmasının doğrudan uygulamasıdır (bkz. 20-Generative\_AI\_and\_Prompt\_Engineer modülü).
4. Çip Tasarımı: Google DeepMind'ın AlphaChip projesi, çip yerleşim planlarını (chip floorplanning) pekiştirmeli öğrenme ile optimize eder.
5. Enerji ve Kaynak Optimizasyonu: Elektrik şebekelerinde talep tahminine dayalı dinamik fiyatlandırma ve yük dengeleme politikalarının öğrenilmesi.

Bu üç dosyayla (21.1.1, 21.2.1, 21.3.1) pekiştirmeli öğrenmenin temel kavramlarından, tablo tabanlı Q-Learning'e, oradan da fonksiyon yaklaşıklamalı politika gradyanı yöntemlerine kadar uzanan yolu tamamladık. Sırada, öğrendiklerinizi pekiştirmek için 21.4.1'deki genel tekrar soruları var.